

# Implémentation from scratch d'un MLP et moteur d'autodifférenciation appliqués à MNIST

Jianye Shi, Hao Qian, Xiang Bian, Abdenmour Boulmis

M1 CHPS – Université Paris-Saclay – UVSQ 2025/2026

*Encadrant:* Aurélien Delval

*Responsables:* Aurélien Delval / Hugo Bolloré / Pablo Oliveira

10 janvier 2026

# Plan de la présentation

- 1 Introduction et Contexte
- 2 Fondements théoriques
- 3 Implémentation
- 4 Expérimentations
- 5 Conclusion et Perspectives

## Déroulement

- Contexte et problématique (6 min)
- Architecture et implémentation (5 min)
- Résultats expérimentaux (8 min)
- Conclusion et perspectives (1 min)

# Introduction et Contexte

## Objectifs

- Implémenter **from scratch** un réseau de neurones (MLP) en C++
- Développer un moteur de différenciation automatique
- Appliquer au dataset MNIST (reconnaissance de chiffres manuscrits)
- Double focus : **Machine Learning + High Performance Computing**

## Aspect ML

- Rétropropagation
- Optimisation (SGD)
- Convergence

## Aspect HPC

- Optimisation bas niveau
- Parallélisme (OpenMP)
- BLAS, vectorisation

# Le problème MNIST

- **Dataset classique** de classification
- Images  $28 \times 28$  pixels (784 features)
- 10 classes (chiffres 0-9)
- 60,000 images d'entraînement
- 10,000 images de test

**Notre approche** : MLP simple mais complet

- Architecture :  $784 \rightarrow 128 \rightarrow 10$
- Activation ReLU
- Loss CrossEntropy



Objectif :  $\sim 98\%$  de précision

# Fondements théoriques

# Architecture du MLP

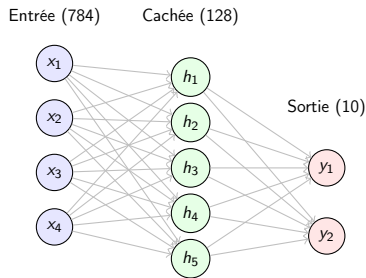


Figure – Structure simple avec **une seule** couche cachée

## Propagation avant (Forward)

Entrée :  $X \in \mathbb{R}^{N \times 784}$

Couche cachée :

$$H = \sigma(XW_1 + b_1)$$

Sortie :

$$Y = \text{softmax}(HW_2 + b_2)$$

## Calcul par mini-batch

- Vectorisation des opérations
- Exploitation du parallélisme
- $N$  exemples simultanément

# Composants Mathématiques du Modèle

## Activation ( $\sigma$ ) : ReLU

$$f(x) = \max(0, x)$$

- Introduit la non-linéarité
- Gradient simple (0 ou 1)
- Évite la saturation

## Coût ( $L$ ) : CrossEntropy

$$L = - \sum y_i \log(\hat{y}_i)$$

- Adaptée à la classification
- Pénalise les erreurs confiantes

## Optimisation : SGD

$$W \leftarrow W - \eta \nabla_W L$$

- $\eta$  : Taux d'apprentissage
- Mise à jour vers le minimum



# Transition : Pourquoi l'Autodifférenciation ?

## Le défi du gradient manuel

- Pour un réseau simple (1 couche), dériver à la main est faisable.
- **Problème** : Avec  $N$  couches, la Règle de la Chaîne devient ingérable.

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial Y} \frac{\partial Y}{\partial H_n} \cdots \frac{\partial H_2}{\partial H_1} \frac{\partial H_1}{\partial W_1}$$

- Chaque modification d'architecture (activation, couche) obligerait à refaire les maths.

## La solution : Différenciation Automatique

- Décomposer le calcul en opérations élémentaires (Graphe).
- Appliquer la Règle de la Chaîne localement et automatiquement.
- **Gain** : Flexibilité totale de l'architecture sans recoder les gradients.

## Principe du DAG (Directed Acyclic Graph)

- Chaque opération = un nœud
- Stockage des valeurs intermédiaires
- Calcul automatique des gradients

## Règle de la chaîne

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial x}$$

## Backward pass

- Tri topologique du graphe
- Propagation des gradients
- Mise à jour des paramètres

## Exemple simple

$$z = x \cdot w$$

$$y = z + b$$

$$L = y^2$$

Gradients :

$$\frac{\partial L}{\partial y} = 2y$$

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial y}$$

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot x$$

# Implémentation

## Choix de conception

- Stockage contigu : `std::vector<double>`
- Optimisation de la localité cache
- Support de la vectorisation (SIMD)

## Implémentations `matmul`

- Naïve (*ijk*) : pédagogique
- Réordonnée (*ikj*) : cache-friendly
- Bloquée : optimisation cache
- OpenMP : parallélisation
- BLAS : référence optimisée

Sélection runtime : `MATMUL_IMPL`

## Initialisation des poids

- **He** : pour ReLU  
 $\text{Var}(W) = 2/n_{in}$
- **Xavier** : pour Sigmoid/Tanh  
 $\text{Var}(W) = 1/n_{in}$
- **Manual** :  $[-0.1, 0.1]$

## Reproductibilité

Graine contrôlée (`-seed`)

# Moteur d'autodiff : Node et MathOps

## Implémentation via Lambdas C++

- `std::shared_ptr<Node>` : gestion mémoire automatique
- Capture du contexte par closures
- Pas de classes dédiées par opération

```
// La puissance des Lambdas pour l'Autodiff (math_ops.cpp)
// Capture du contexte {a, b} par closure
node->setBackwardFn([a, b](const Matrix& grad) {
    // Application directe de la Chain Rule
    a->addGrad(grad.mul(b->value())); // dL/da = grad * b
    b->addGrad(grad.mul(a->value())); // dL/db = grad * a
});
```

## Avantages

- Tri topologique (DFS)
- Broadcasting automatique

## Performance

- Sans récursion profonde
- Typage via `OpKind`

# Construction du modèle MLPNetwork

Le réseau est construit par composition modulaire avec support multi-couches :

```
// Construction flexible (Support multi-couches)
int prev = input_dim;
for (int h : hidden_dims) {
    // Couche cachee + Activation (ReLU/Sigmoid...)
    net.addLayer(makeLinear(prev, h), makeActivation(act));
    prev = h;
}
// Couche de sortie (Logits bruts)
net.addLayer(makeLinear(prev, output_dim), nullptr);
```

**Exemple :** -hidden\_sizes 256,128,64 cree 784->256->128->64->10

# Pipeline d'entraînement

## Modules principaux

- `LinearLayer` :  $y = xW + b$
- `ActivationFunction`  
ReLU, Sigmoid, Tanh
- `LossFunction`  
MSE, CrossEntropy
- `Optimizer` : SGD

## Boucle Trainer

- 1 Charger mini-batch ( $X, Y$ )
- 2 **Forward** :  $\hat{Y} = \text{Model}(X)$
- 3 **Loss** :  $L = \text{Loss}(\hat{Y}, Y)$
- 4 **Backward** : calcul gradients
- 5 **Update** :  $W \leftarrow W - \eta \nabla W$

```
// Implementation SGD (optimizer.cpp)
void SGD0Optimizer::step() {
    for (auto& p : parameters_) {
        Matrix& val = const_cast<Matrix&>(p->value());
        for(size_t i=0; i<val.data.size(); ++i)
            val.data[i] -= lr_ * p->grad().data[i];
    }
}
```

# Expérimentations



## Matériel

- AMD Ryzen 9 8940HX
- 32 cœurs / 64 threads
- 30 GiB RAM
- Cache L3 : 64 MiB

## Logiciel

- Fedora Linux 42
- GCC 15.2.1
- Flags : `-O3 -march=native`

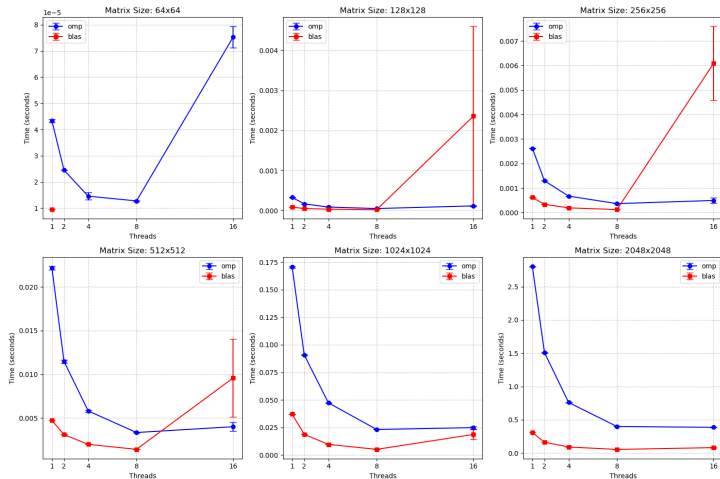
## Protocole rigoureux

- **Fréquence verrouillée** : 4.0 GHz
- **Affinité CPU** : taskset
- **Warm-up** : 5 itérations
- **Graines fixées** : moyenne sur 3 runs (42, 43, 44)

## Objectif

Mesures fiables et reproductibles pour comparer les optimisations

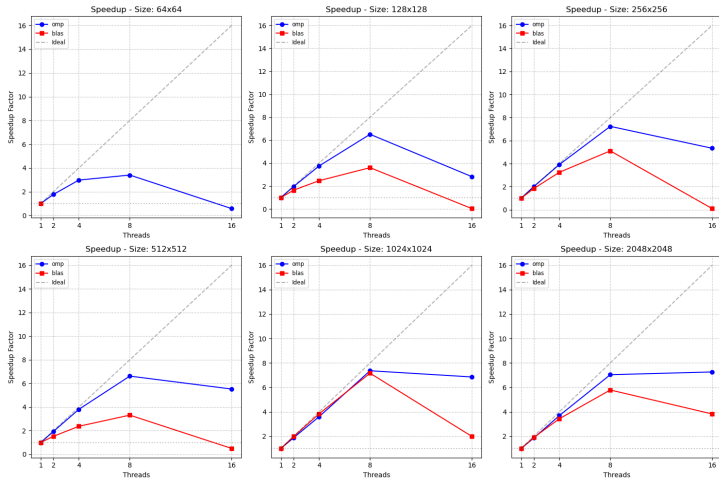
# Passage à l'échelle : Temps d'exécution



**Analyse** : À 64x64, le surcoût de gestion des threads dépasse le gain de calcul au-delà de 8 threads.

**Conclusion** : La parallélisation n'est efficace que si la charge de calcul est suffisante pour masquer

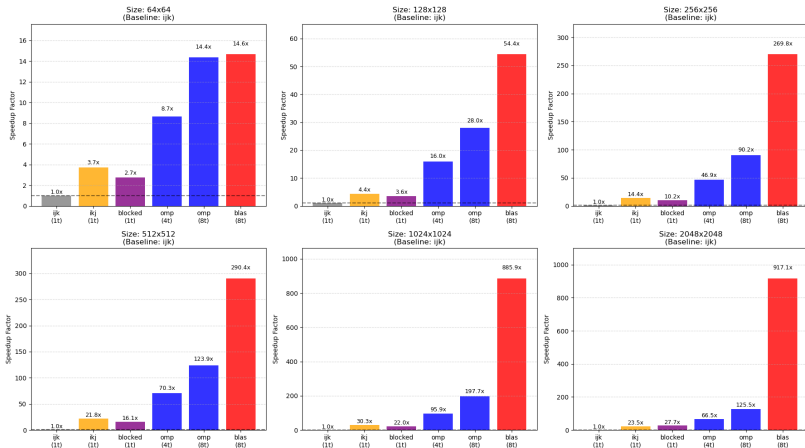
# Passage à l'échelle : Accélération (Speedup)



**Analyse** : L'accélération est quasi-linéaire jusqu'à 8 threads (cœurs physiques), puis sature.

**Conclusion** : L'HyperThreading n'apporte pas de gain pour ce type de charge CPU-bound (FPU)

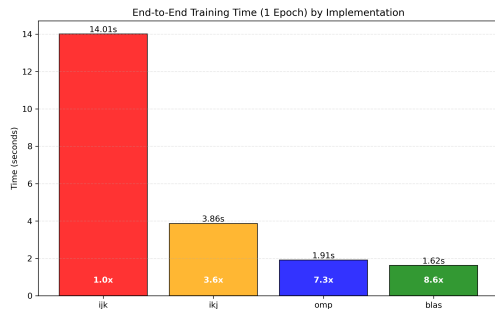
# Hiérarchie : Impact des optimisations sur les performances



**Observation :** Les gains de performance augmentent fortement avec la taille du problème.

**Conclusion :** Les performances à grande échelle reposent sur une exploitation efficace de la hiérarchie mémoire.

# Profiling et Loi d'Amdahl



## Analyse

- Naïf : **96%** du temps dans matmul (Compute-bound)
- BLAS : matmul négligeable ( $< 1\%$ )

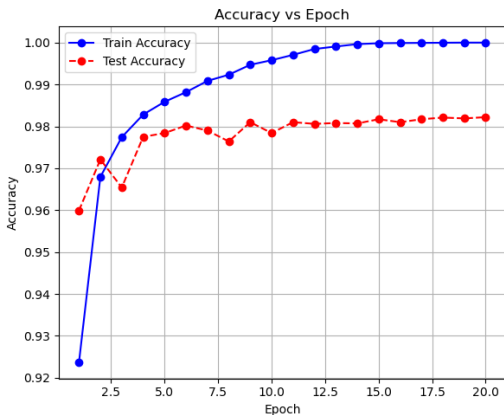
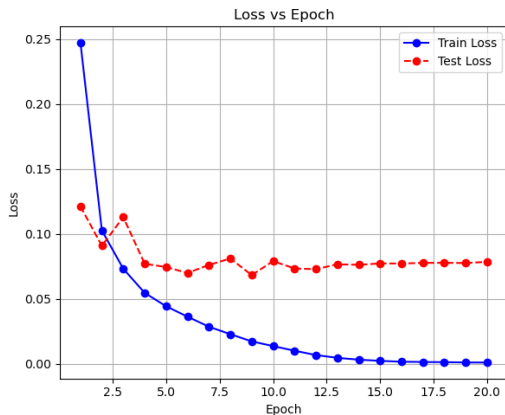
## Conclusion

- Goulot d'étranglement  $\rightarrow$  **E/S et allocations mémoire.**
- Gain total limité par loi d'Amdahl

Speedup global

**7.6×**  
(limité par la loi d'Amdahl)

# Analyse de la sensibilité au learning rate



**Observation :** La loss converge rapidement vers 0 (train), tandis que la validation stabilise (plateau).  
*L'accuracy d'entraînement continue de monter alors que la validation plafonne.*

**Précision finale**

~98.2%

**Convergence**

Rapide et stable  
(LR=0.01, ReLU)

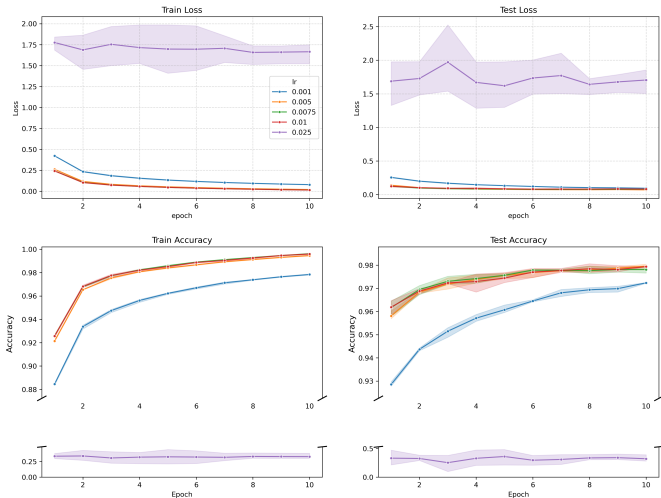
**Généralisation**

Surapprentissage contrôlé  
(Hidden=128)

*Bon compromis capacité / généralisation validé par Grid Search*

# Impact du Learning Rate

Learning Rate Experiment Results (10 Epochs)



## Analyse

- LR=0.001 : Trop lent
- LR=0.025 : Instable (variance forte)

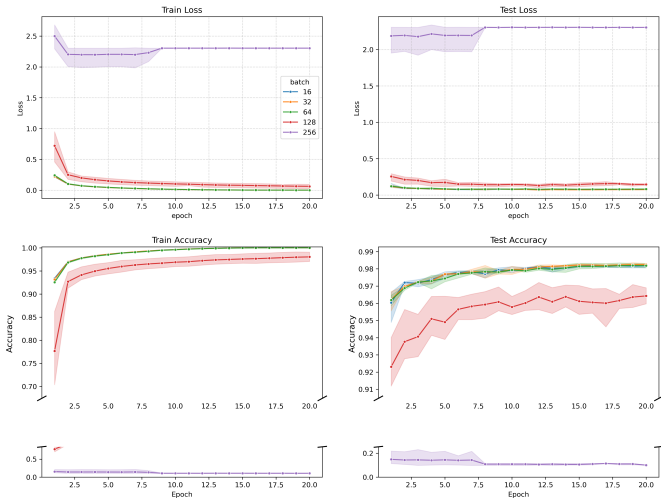
## Conclusion

- Un LR modéré [0.005, 0.01] offre le meilleur compromis vitesse/stabilité.



# Impact du Batch Size

Batch Size Experiment Results (20 Epochs)



## Analyse

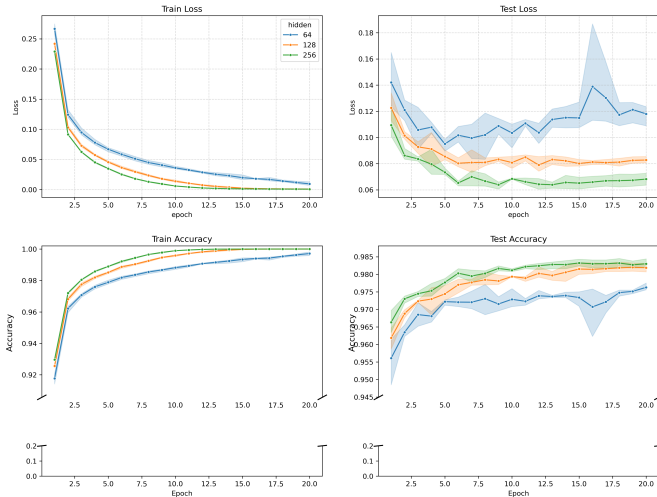
- Petits (16-32) : Lents
- Grands (128+) : Apparition d'un écart train-test, signe de surapprentissage

## Conclusion

- Batch=64 maximise le débit sans sacrifier la convergence (à nombre d'époques fixé).

# Impact de la Taille Cachée

Hidden Size Experiment Results (20 Epochs)



## Analyse

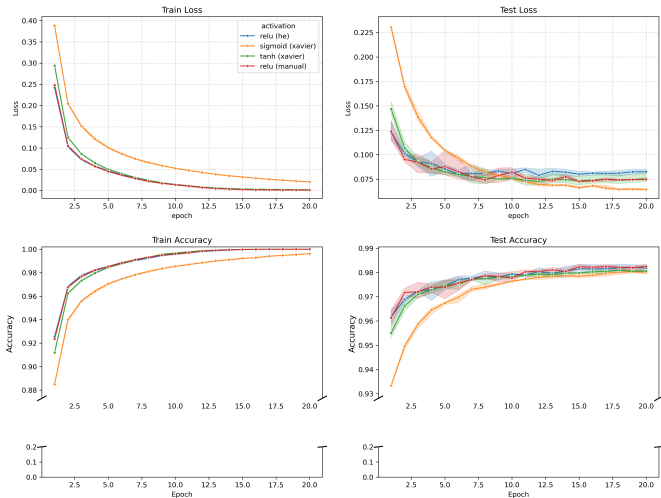
- H=64 : Sous-apprentissage
- H=256 : Pas de gain

## Conclusion

- H=128 suffit pour capturer la complexité de MNIST sans surcoût inutile.

# Impact de l'Activation

Activation Function Experiment Results (20 Epochs)



## Analyse

- Sigmoid : La moins performante (saturation).
- Autres : Résultats similaires et satisfaisants.

## Conclusion

- Peu de variation globale, sauf pour la Sigmoid qui décroche nettement.

### Configuration finale

Learning Rate : **0.01**  
Batch Size : **64**  
Hidden Size : **128**  
Activation : **ReLU**  
Initialisation : **He**

**Précision :  $\sim 98.2\%$**

# Conclusion et Perspectives

## Réalisations

- ✓ Implémentation complète **from scratch** en C++
- ✓ Moteur d'autodiff fonctionnel et élégant (DAG + lambdas)
- ✓ Validation ML : **~98.2%** de précision sur MNIST
- ✓ Optimisation HPC : speedup **200×** sur le noyau de calcul

## Enseignements clés

- **Dual challenge** : algorithmique (convergence) + computationnel (performance)
- **Loi d'Amdahl** : après optimisation du calcul, les I/O deviennent le goulot
- **Importance du protocole** : mesures reproductibles essentielles

## Extensions ML

- Optimiseurs avancés (Momentum, Adam)
- Régularisation (L2, Dropout)
- Architectures CNN
- Datasets plus complexes

## Optimisations HPC

- Support GPU (CUDA)
- Vectorisation explicite (AVX-512)
- Memory pool / arenas

## Parallélisme de Données & Entraînement

- **Data Parallelism (MPI)** :  
Traitement distribué sur plusieurs nœuds.
- **Batch Gradient Descent** :  
Parallélisme de haut niveau sur les lots.
- **Réseaux Indépendants** :  
Entraînement parallèle + Synchronisation périodique.

## Reproductibilité

- Framework de benchmarking
- CI/CD pour validation
- Documentation utilisateur

# Merci de votre attention !

Questions ?



# Architecture logicielle

Haut niveau

**main.cpp** – Point d'entrée CLI

**Trainer** – Boucle forward → backward → update

**Loss + Optimizer** – CrossEntropy, SGD

**MLPNetwork** – Assemblage multi-couches

**LinearLayer + Activation** – Composants réseau

**Node + MathOps** – Moteur d'autodifférenciation

Bas niveau

**Matrix/Tensor** – Opérations mathématiques bas niveau

## Implémentation du produit matriciel

```
// Version naive (ijk) - mauvaise localite cache
for (int i = 0; i < m; ++i)
    for (int j = 0; j < p; ++j)
        for (int k = 0; k < n; ++k)
            C(i,j) += A(i,k) * B(k,j);

// Version reordonnee (ikj) - meilleure localite
for (int i = 0; i < m; ++i)
    for (int k = 0; k < n; ++k)
        for (int j = 0; j < p; ++j)
            C(i,j) += A(i,k) * B(k,j);

// Version OpenMP
#pragma omp parallel for
for (int i = 0; i < m; ++i)
    for (int k = 0; k < n; ++k)
        for (int j = 0; j < p; ++j)
            C(i,j) += A(i,k) * B(k,j);
```

## Exemple de calcul de gradient (LinearLayer)

$$\text{Forward : } Y = XW + b$$

$$\text{Backward : } \frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} \cdot W^T$$

$$\frac{\partial L}{\partial W} = X^T \cdot \frac{\partial L}{\partial Y}$$

$$\frac{\partial L}{\partial b} = \sum \frac{\partial L}{\partial Y}$$

## Implémentation

- Stockage de  $X$  pendant le forward
- Calcul des trois gradients lors du backward
- Accumulation dans les nœuds parents

## MNISTDataset

- Lecture binaire IDX
- Normalisation  $[0, 1]$
- Labels one-hot encoded

## DataLoader

- Charge MNIST (60k/10k)
- Génère mini-batches  $(X, Y)$
- Shuffle (seed contrôlé)
- `reset()`, `nextBatch()`

## main.cpp (CLI)

- Parse arguments
  - epochs, -lr, -batch\_size
  - hidden\_sizes, -activation
  - init, -seed
- Crée : Dataset, DataLoader, MLPNetwork, Loss, Optimizer, Trainer
- Boucle d'entraînement par époques
- Sauvegarde métriques CSV